

Cookbook

Cookbook Personal Garmin AI Coach

Zur Veranstaltung ‚New Trends in IT und Management der Digitalen Transformation‘ im Studiengang ‚Bachelor Informatik Telekom‘

vorgelegt dem Fachbereich
Informatik und Wirtschaftsinformatik

der

Provadis School of International Management and Technology

von

Khalid Lakniti

D837

BIN_T23_F2

khalid.lakniti@stud-provadis-hochschule.de

Erstgutachter: Prof. Dr. Richard Beetz

Ende der Bearbeitungsfrist: 11.08.2026

Inhalt

Abbildungsverzeichnis.....	iv
Tabellenverzeichnis	v
Code-Verzeichnis	vi
1 Ziele, Anwendungsfall und Begründung	1
1.1 Der Anwendungsfall	1
1.2 Begründung: Warum diese Lösung sinnvoll und überlegen ist	1
2 Kurzskeizze der Lösung.....	3
2.1 Architekturüberblick	3
2.2 Technologie-Stack	3
3 Setup & Konfiguration.....	4
3.1 Voraussetzungen (Python 3.11, Keys).....	4
3.2 Environment-Variablen	5
3.3 Quick Start (lokal)	6
3.3.1 Abhängigkeiten installieren	6
3.3.2 .env anlegen	6
3.3.3 Dashboard starten:	6
3.3.4 Im Browser öffnen	6
3.3.5 Registrieren & verifizieren	6
4 Datenbeschaffung & Persistenz.....	7
4.1 Garmin-Login, MFA & Backoff.....	7
4.2 Persistenz als JSON	8
4.3 Datenmodelle & Schemata (daily_stats, activities, profile).....	8
5 Das "Gehirn": Der Coach Agent	10
5.1 Architektur des Agents (ETLA)	10
5.2 Der System-Prompt.....	11
5.3 Aufbau des User-Payloads & Rules.....	12
5.4 Bilinguale Inferenz	12
5.5 Antwort Verarbeitung	13
6 Guardrails & Fallback-Strategie.....	13
6.1 Zwei Recovery-Stufen	13
6.2 Deterministische Fallbacks	14
6.3 Goal-basierte Intensität	15
7 Wetter-Integration & Context-Awareness	15
7.1 Open-Meteo Geocoding & Wetter	15
7.2 Indoor/Outdoor-Entscheidung	16

8	Resilienz: Caching, Rate-Limitierung & Retry	17
8.1	Drei getrennte Caching-Ebenen.....	17
8.2	Persistierter Backoff für Garmin.....	17
8.3	Retry für das LLM	17
9	Sicherheit & Datenschutz.....	18
9.1	HashiCorp Vault (KV v2).....	18
9.2	Bedrohungsmodell & bewusste lokale JSON-Ablage	19
10	Benutzeroberfläche & Scheduling.....	20
10.1	Streamlit-Dashboard.....	20
10.2	Background-Worker-Thread (60s-Loop)	21
10.3	Onboarding & 6-stellige Verifizierung	22
11	Deployment & Betrieb.....	23
11.1	Docker	24
11.2	Systemd und Betrieb in der Cloud.....	24
12	Testing & Lokaler Trockenlauf.....	25
12.1	Coach-Logik isoliert ausführen	25
12.2	Garmin-Abruf gezielt testen	26
12.3	Manuelle Testwerte im Dashboard.....	26
12.4	Stand der automatisierten Tests.....	26
13	Fazit und Lessons Learned	27
13.1	Zusammenfassung	27
13.2	Lessons Learned	27
13.3	Ausblick.....	28
14	Screenshots	29

Abbildungsverzeichnis

Abbildung 1: Symbolbild des Coaches.....	2
Abbildung 2: Systemarchitektur	3
Abbildung 3: Manual Entry	9
Abbildung 4: Sequenzdiagramm	11
Abbildung 5: E-Mail Notification mit „Rest Day“	14
Abbildung 6: Garmin & Vault	19
Abbildung 7: Dashboard Overview	21
Abbildung 8: Login/Register Screen	23
Abbildung 9: Registration Code via E-Mail	23
Abbildung 10: Recent Activities.....	29
Abbildung 11: Discord Embed	30

Tabellenverzeichnis

Tabelle 1: Technologie-Stack	4
Tabelle 2: technische Voraussetzungen	5
Tabelle 3: Environment-Variablen	5
Tabelle 4: Garmin Fehlerklassen	7
Tabelle 5: Backoff-Staffelung	8
Tabelle 6: LLM-Parameter	13
Tabelle 7: Recovery-Stufen	14
Tabelle 8: Goal-Intensität	15
Tabelle 9: Open-Meteo-Felder.....	16
Tabelle 10: Indoor/Outdoor-Logik	16
Tabelle 11: drei Caching-Ebenen	17
Tabelle 12: Umgebungsvariablen der Vault-Anbindung.....	18
Tabelle 13: Ablageorte und Schutzniveau der verschiedenen Datenarten	20
Tabelle 14: Funktionsblöcke des Dashboards	21
Tabelle 15: Eckdaten des Docker-Images	24
Tabelle 16: Eckdaten des Systemd-Dienstes	25
Tabelle 17: Kommandozeilenoptionen des Coach Agent.....	25
Tabelle 18: Kommandozeilenoptionen des Garmin-Abrufs.....	26

Code-Verzeichnis

1: daily_stats.json	9
2: activities.json	10
3: user_profile.json	10
4: COACH_SYSTEM_PROMPT	12

1 Ziele, Anwendungsfall und Begründung

1.1 Der Anwendungsfall

Der "Personal Garmin AI Coach" ist ein autonomer Agent, der Gesundheits- und Aktivitätsdaten aus Garmin Connect extrahiert, diese mit lokalen Umweltdaten (Wetter) anreichert und mithilfe eines Large Language Models (LLM) eine konkrete, kontextbezogene Trainingsempfehlung für die nächste Einheit erstellt.

Im Gegensatz zu klassischen Trainingsplänen erzeugt das System bewusst keinen Wochenplan, sondern beantwortet die Frage: „Was soll ich basierend auf meinem heutigen Erholungszustand und dem aktuellen Wetter als Nächstes trainieren?" Die Empfehlung wird automatisiert über Discord oder E-Mail zugestellt.

1.2 Begründung: Warum diese Lösung sinnvoll und überlegen ist

Herkömmliche Fitness-Tracker und statische Trainingspläne leiden unter zwei massiven Problemen:

1. **Mangelnde Adaption an den "Ist-Zustand":** Ein klassischer Plan ignoriert, wenn der Athlet schlecht geschlafen hat oder das autonome Nervensystem überlastet ist (niedrige *Body Battery*).
2. **Ignoranz gegenüber physischen & lokalen Constraints:** Standard-Apps gehen oft von idealen Bedingungen und Standard-Mobilität aus (z. B. keine Berücksichtigung von Rollstuhlnutzung oder schlechtem Wetter).

Dieses Projekt löst die Probleme durch einen echten "Closed-Loop"-Ansatz. Der Agent agiert als dynamischer Filter. Er kombiniert harte Rohdaten (Pace, Training Effect, akute Trainingslast) mit biometrischen Erholungsdaten (Sleep Score, Body Battery, Stress) und externen Umweltdaten (Open-Meteo). Diese aggregierten Daten werden einem LLM übergeben, das durch präzises Prompt-Engineering und vorgelagerte, deterministische Sicherheitsregeln dazu gebracht wird, spezifische Constraints (z. B. Rollstuhlnutzung → kein Lauf-Intervall) und Wetterbedingungen (z. B. Starkregen → Indoor-Workout) zu berücksichtigen.

Der entscheidende Mehrwert liegt in der Kombination aus KI-Flexibilität und deterministischen Guardrails: Das LLM formuliert die Empfehlung, aber harte Regeln im Code stellen sicher, dass bei kritischen Erholungswerten niemals eine Überlastung empfohlen wird (siehe Kapitel 6).



Abbildung 1: Symbolbild des Coaches

2 Kurzskeizze der Lösung

2.1 Architekturüberblick

Die Lösungsarchitektur ist modular aufgebaut und trennt strikt zwischen Datengewinnung, Logik und Präsentation.

- **Ingestion (Datenbeschaffung):** Ein Python-Backend (garminconnect) ruft historische und tagesaktuelle Metriken ab. Um Rate-Limits (HTTP 429) zu umgehen, wurde ein Retry-Mechanismus mit Exponential-Backoff implementiert.
- **Context & Environment:** Die Geodaten des Nutzers werden via Open-Meteo aufgelöst, um das aktuelle Wetter in den Kontext der Entscheidung einzubeziehen.
- **Intelligence (Agent):** Das System nutzt Groq Cloud für die LLM-Inferenz. Das Modell wird über den JSON-Mode und einen strukturierten Prompt gezwungen, ein bilinguales JSON-Objekt (Englisch + Deutsch) in einem einzigen Aufruf zurückzugeben.
- **Persistence & Security:** Nutzerprofile, Metriken und Caches werden als JSON persistiert. Sensible Login-Daten (Garmin) werden in HashiCorp Vault OSS (KV v2 Engine) verschlüsselt abgelegt.
- **Interface & Scheduling:** Ein Streamlit-Dashboard fungiert als Konfigurationszentrale. Ein in den Streamlit-Prozess integrierter Background-Worker-Thread triggert die Empfehlungen zu zwei konfigurierbaren Tageszeiten.

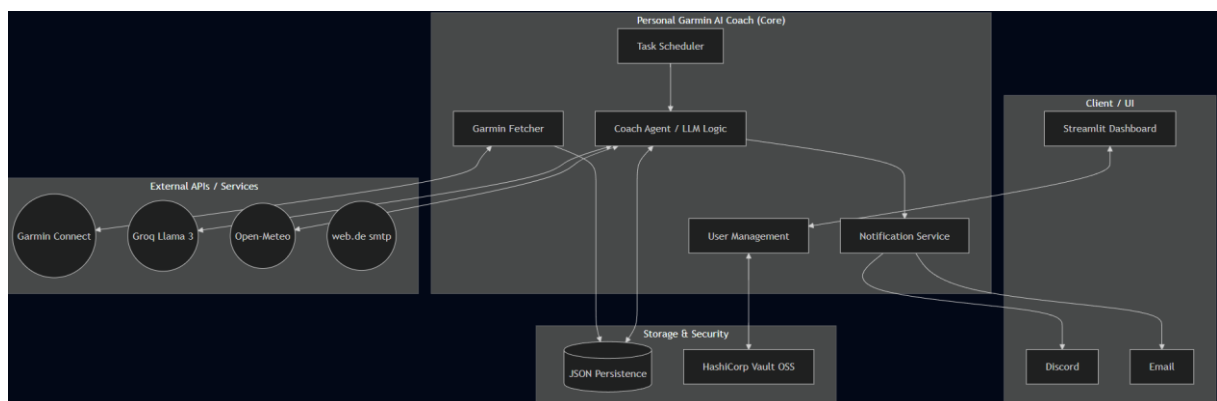


Abbildung 2: Systemarchitektur

2.2 Technologie-Stack

Ebene	Technologie / Tool	Zweck
Sprache	Python 3.11	Kern der Anwendung
Frontend	Streamlit	Dashboard & Benutzerkonfiguration
Intelligence	Groq Cloud – Llama 3.3 70B (llama-3.3-70b-versatile)	LLM für die Coaching-Logik
API Wrapper	garminconnect	Interface zu Garmin Connect
Security	HashiCorp Vault OSS (KV v2)	Sichere Verwahrung von Secrets
Infrastructure	Docker & Systemd	Deployment und Autostart
Weather	Open-Meteo (Geocoding + Forecast)	Kontext-Informationen (Umgebung)
Notifications	Discord (Bot) & SMTP	Push-Benachrichtigungen & HTML-Mails

Tabelle 1: Technologie-Stack

3 Setup & Konfiguration

Dieses Kapitel beschreibt alles, was zum lokalen Nachbau und Start der Anwendung notwendig ist. Ziel ist, dass ein Leser das Projekt von Null bis zum laufenden Dashboard bringen kann.

3.1 Voraussetzungen (Python 3.11, Keys)

Komponente	Anforderung	Pflicht?
Python	3.11 (im Docker-Image: python:3.11-slim)	Ja
Garmin-Connect-Konto	E-Mail + Passwort	Ja (für echte Daten)
Groq API Key	Account bei Groq Cloud	Ja (für KI-Empfehlungen)
Discord Bot Token	Discord-Applikation mit Bot	Optional (für DM-Versand)
SMTP-Zugang	z. B. web.de / GMX	Optional (für E-Mail)
HashiCorp Vault OSS	laufende Vault-Instanz, KV v2	Optional (für sichere Credential-Ablage)

Tabelle 2: technische Voraussetzungen

Die Python-Abhängigkeiten sind bewusst schlank gehalten (requirements.txt):

```
garminconnect
python-dotenv
instructor
streamlit
groq
```

3.2 Environment-Variablen

Sämtliche Secrets und externen Endpunkte werden über eine `.env`-Datei im Projekt-Root geladen (via `python-dotenv`). Die folgende Tabelle listet alle im Code tatsächlich ausgewerteten Variablen inklusive ihrer Default-Werte:

Variable	Zweck	Pflicht?
GARMIN_EMAIL	Garmin-Login (Fallback, wenn nicht in Vault)	Ja*
GARMIN_PASSWORD	Garmin-Passwort (Fallback, wenn nicht in Vault)	Ja*
GROQ_CLOUD_KEY	API-Key für die LLM-Inferenz	Ja
DISCORD_BOT_TOKEN	Token zum Versand von Discord-DMs	Optional
DISCORD_SERVER_INVITE	Invite-Link im Onboarding	Optional
MAIL_USERNAME	SMTP-Benutzer für E-Mail-Versand	Optional
MAIL_PASSWORD	SMTP-Passwort	Optional
VAULT_ADDR	Adresse der Vault-Instanz	Optional
VAULT_TOKEN	Vault-Zugriffstoken	Optional
VAULT_KV_PATH	Pfad der KV-v2-Engine	Optional

Tabelle 3: Environment-Variablen

* *Garmin-Credentials können entweder aus der `.env` (Fallback) oder bevorzugt aus HashiCorp Vault geladen werden. Im Code gilt: erst Vault/gespeicherte Credentials, dann `.env`.*

Die `.env` ist in der `.gitignore` ausgeschlossen und darf niemals eingecheckt werden. Garmin-Passwörter gehören produktiv in Vault, nicht in die `.env` (siehe [Kapitel 9](#)).

3.3 Quick Start (lokal)

3.3.1 Abhängigkeiten installieren

```
pip install -r requirements.txt
```

3.3.2 .env anlegen

Siehe [Kapitel 3.2](#)

3.3.3 Dashboard starten:

```
streamlit run web/app.py
```

3.3.4 Im Browser öffnen

(lokal Standard-Port 8501; im Docker-Container 8080).

3.3.5 Registrieren & verifizieren

E-Mail oder Discord wählen → 6-stelligen Code anfordern → Code eingeben → Dashboard wird freigeschaltet (Details in Kapitel 10.3).

Port-Hinweis: Lokal nutzt Streamlit standardmäßig **8501**. Der Docker-Container exponiert hingegen **8080** (siehe Dockerfile in [Kapitel 11](#)).

4 Datenbeschaffung & Persistenz

4.1 Garmin-Login, MFA & Backoff

Die Datenbeschaffung erfolgt über die Library `garminconnect`, die einen Web-Login bei Garmin Connect simuliert. Das zugehörige Skript (`fetch_garmin_data.py`) ist als eigenständig aufrufbares CLI gebaut und lässt sich pro Nutzer scopen:

```
python core/fetch_garmin_data.py --user-id <ID> [--force] [--debug]
```

Beim Login werden mehrere Fehlerklassen gezielt unterschieden und protokolliert:

Fehlerfall	Erkennung im Code	Reaktion
Falsche Zugangsdaten	GarminConnectAuthenticationError, Auth-Hinweis in Log-Capture	auth_error, Abbruch
Rate-Limit	"429" / "rate limit" in der Fehlermeldung	rate_limit_429 → Backoff
CAPTCHA verlangt	"captcha" in der Fehlermeldung	captcha_required
Verbindungsfehler	GarminConnectConnectionError	Logging, ggf. Retry

Tabelle 4: Garmin Fehlerklassen

Statt eines blockierenden Wartens nutzt das System einen persistierten, gestaffelten Backoff (`fetch_garmin_data.py:93`). Nach wiederholten Fehlversuchen wird der nächste erlaubte Ab-rufzeitpunkt gespeichert:

Fehlversuch	Nächster Versuch frühestens in
1. Fehler	1 Stunde
2. Fehler	4 Stunden
3. Fehler	24 Stunden
4.+ Fehler	48 Stunden

Tabelle 5: Backoff-Staffelung

4.2 Persistenz als JSON

Alle Daten werden als JSON im data-Verzeichnis abgelegt. Die Persistenzschicht (`data_persistence.py`) ist konsequent nutzer-scoped: Jeder Nutzer erhält ein eigenes Verzeichnis unter `data/users/<user_id>/`. Der `user_id`-Wert wird dabei über `_safe_user_segment()` dateisystemsicher bereinigt (nur a–z, 0–9, `_`, `-`).

```
data/
├── coach_recommendation.json      # globaler Fallback-Cache
├── users/
│   └── <user_id>/
│       ├── daily_stats.json      # bis zu 7 Tage Gesundheitsmetriken
│       ├── activities.json       # letzte Aktivitäten
│       └── user_profile.json     # Profil, Ziele, Notification-Settings
```

Wichtige Eigenschaften der Persistenz:

- **Merge statt Überschreiben:** `save_daily_stats()` lädt bestehende Daten und merged neue Tageswerte hinein (heutige Werte überschreiben den jeweiligen Tag).
- **Robustes Lesen:** Defekte/leere JSON-Dateien werden über `try/except json.JSONDecodeError` abgefangen und als leeres Objekt behandelt. Kein Crash bei korrupten Dateien.
- **Gezieltes Löschen:** Einzelne Tage (`delete_daily_stat`) oder Aktivitäten (`delete_activity`) können entfernt werden.

Hinweis: Das gesamte data-Verzeichnis ist in der `.gitignore` ausgeschlossen, da es personenbezogene Gesundheitsdaten enthält.

4.3 Datenmodelle & Schemata (`daily_stats`, `activities`, `profile`)

Damit das Projekt nachvollziehbar nachgebaut werden kann, folgen die drei zentralen Datenstrukturen mit echten (anonymisierten) Beispielen.

`daily_stats.json` - ein Objekt, dessen Schlüssel das Datum (YYYY-MM-DD) ist:

```
{
  "2026-05-05": {
    "sleep_score": 75,
    "body_battery": 70,
    "stress": 20,
```

```

    "vo2_max": 45.0,
    "resting_heart_rate": 60,
    "training_load_acute": 0.0,
    "training_balance_feedback": "N/A",
    "time": "08:51:00",
    "source": "manual"
  }
}

```

1: daily_stats.json

Das Feld source unterscheidet "garmin" (automatisch abgerufen) von "manual" (manuelle Eingabe im Dashboard).



Abbildung 3: Manual Entry

activities.json - ein Wrapper-Objekt mit Zeitstempel und Aktivitätsliste (fetch_garmin_data.py:609):

```

{
  "last_updated": "2026-05-10T10:49:55",
  "activities": [
    {
      "id": "1234567890",
      "index": 0,
      "date": "2026-05-10",
      "time": "10:49:55",
      "activity_type": "running",
      "primary_metric": 3.2,

```

```
    "duration": 1929,  
    "calories": 472,  
    "distance": 6530,  
    "source": "garmin"  
  }  
]  
}
```

2: activities.json

Besonderheit: `primary_metric` ist kontextabhängig, bei Ausdaueraktivitäten der Training-Effect-Score, bei Krafttraining die Übungsinformation. Für Kraft-/Gewichtstraining wird das Feld `distance` bewusst weggelassen, da es dort nicht sinnvoll ist.

`user_profile.json` - Profil und Benachrichtigungs-Einstellungen pro Nutzer:

```
{  
  "last_updated": "2026-05-01T12:37:11.396544",  
  "profile": {  
    "mobility": "Healthy",  
    "preference": "",  
    "goal": "Strength Focus",  
    "notify_discord": true,  
    "discord_user_id": "324607583841419276"  
  }  
}
```

3: user_profile.json

Die drei Felder `mobility`, `preference` und `goal` bilden direkt das `CoachProfile` (`coach_agent.py:35`), das in Kapitel 5 den Prompt steuert.

5 Das "Gehirn": Der Coach Agent

Der Coach Agent (`coach_agent.py`) ist das Herzstück. Er aggregiert die Daten, baut daraus einen strukturierten Prompt, ruft das LLM auf und validiert dessen Antwort.

5.1 Architektur des Agents (ETLA)

Der Datenfluss folgt dem Prinzip Extract – Transform – Load – Action:

1. **Extract:** Roh-JSON via Garmin-Client.

2. **Transform:** Reduktion auf die letzten 7 Tage (`_compact_daily_stats`) und 7 Aktivitäten (`_compact_activities`) - kompaktes, token-sparendes Kontextformat.
3. **Load:** Anreicherung mit Wetterdaten (Open-Meteo, [siehe Kapitel 7](#)).
4. **Action:** Übersetzung der LLM-Entscheidung in ein Discord-Embed oder HTML-Mail.

Dieser Datenfluss ist auch in folgendem Sequenzdiagramm dargestellt:

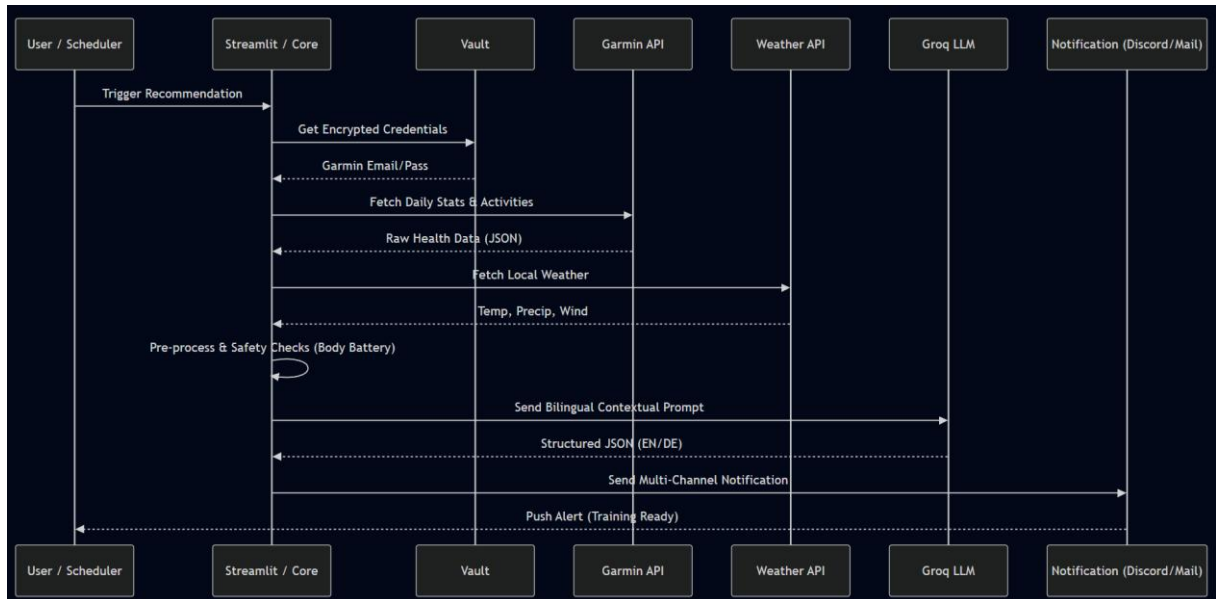


Abbildung 4: Sequenzdiagramm

5.2 Der System-Prompt

Die Steuerung des Modells erfolgt über einen festen System-Prompt, der Format, Sprache und die wichtigsten Sicherheitsregeln vorgibt. Er ist im Code als Konstante `COACH_SYSTEM_PROMPT` hinterlegt (`coach_agent.py:41`):

```

You are a precise fitness coach with ABSOLUTE priority on overload protection.
Reply ONLY as JSON with BOTH English and German versions.
Use these keys: title_en, title_de, recommendation_en, recommendation_de,
alternative_en, alternative_de, intensity, reasoning_en, reasoning_de.
The recommendation MUST describe exactly the next concrete session or at most
the next 1-2 sessions (duration, structure, intensity).
No weekly plans, no routines, no frequency statements like '2 times per week'.
Use concrete numbers and ALWAYS include alternatives as their own keys
(alternative_en / alternative_de).
IMPORTANT: If Body Battery < 50 OR Sleep < 60, then ONLY recovery training
with intensity 1-4.
  
```

The reasoning MUST explicitly reference current health data (Sleep Score, Body Battery, Stress, VO2Max, RHR, last activity).

IMPORTANT: In the reasoning, ALWAYS use the exact goal name from the user profile. No synonyms or paraphrases. No generic phrases.

IMPORTANT: If Body Battery is around 35 or lower, say that no training should be done today and return title 'Rest Day'.

IMPORTANT: In the German translation, use transliteration with ae/oe/ue instead of umlauts.

4: COACH_SYSTEM_PROMPT

Wesentliche Designentscheidungen:

- Bilinguale Ausgabe in einem Call (en/de) spart einen separaten Übersetzungs-Aufruf.
- Nur die nächste Einheit, kein Wochenplan.
- Erholungsschutz direkt im Prompt verankert (Details in Kapitel 6).
- Transliteration (ae/oe/ue) statt Umlaute für robuste Darstellung in Discord/E-Mail.

5.3 Aufbau des User-Payloads & Rules

Zusätzlich zum System-Prompt erhält das Modell ein vorverarbeitetes JSON-Objekt mit dem gesamten Entscheidungskontext (`build_coach_prompt`, `coach_agent.py:401`). Es enthält u. a.:

- `user_profile` (`mobility`, `preference`, `goal`, `goal_intensity_baseline`)
- `health_status` (inkl. abgeleiteter Flags `recovery_critical`, `recovery_rest_day`)
- `training_load`, `weather`, `history_7_days`, `recent_activities`
- eine explizite `rules`-Liste mit harten Vorgaben (z. B. Wetter-Regel, Rollstuhl-Constraint, exakter Ziel-Name)

Die Flags werden deterministisch im Code vorab berechnet und nicht dem Modell überlassen.

5.4 Bilinguale Inferenz

Der eigentliche Aufruf erfolgt über die `GroqCoachClient`-Klasse (`coach_agent.py:508`) mit dem Modell `llama-3.3-70b-versatile`:

Parameter	Wert	Begründung
<code>temperature</code>	0.2	niedrig → konsistente, wenig „kreative“ Empfehlungen

max_tokens	450	begrenzt Kosten/Latenz; reicht für bilinguales JSON
response_format	{"type": "json_object"}	erzwingt valides JSON (JSON-Mode)

Tabelle 6: LLM-Parameter

5.5 Antwortverarbeitung

Die Strukturierung erfolgt dreistufig:

1. **JSON-Mode erzwingen** - Groq liefert dank `response_format={"type": "json_object"}` valides JSON.
2. **Defensives Parsing** - `extract_json_response()` entfernt evtl. Markdown-Fences und extrahiert notfalls den `{ ... }`-Block, falls das Modell zusätzlichen Text liefert.
3. **Key-Normalisierung** - `normalize_recommendation_keys()` stellt sicher, dass immer alle erwarteten Felder vorhanden sind (bilingual *_en/*_de, mit Fallback auf Legacy-Keys). Dadurch finden die nachgelagerten Dienste (Discord/E-Mail) garantiert ihre Felder.

Dieser Ansatz ist pragmatischer als Pydantic, erreicht aber dasselbe Ziel: garantiert vollständige, maschinenlesbare Ausgabe.

6 Guardrails & Fallback-Strategie

Ein KI-Coach, der Gesundheitsdaten verarbeitet, darf niemals eine gefährliche Empfehlung ausgeben. Das System kombiniert daher das LLM mit deterministischen Schutzmechanismen im Code. Drei Ebenen greifen ineinander: Erholungsschutz, deterministischer Fallback und zielbasierte Intensität.

6.1 Zwei Recovery-Stufen

Bevor das LLM überhaupt formuliert, berechnet der Code aus den aktuellen Werten zwei boolesche Flags und übergibt sie als harte Regeln in den Prompt:

Stufe	Bedingung	Konsequenz
Recovery-Modus	Sleep Score < 60 oder Body Battery < 50	nur Erholungstraining, Intensität auf 1–4 begrenzt
Rest Day	Body Battery < 35	kein Training; Titel wird Rest Day

Tabelle 7: Recovery-Stufen

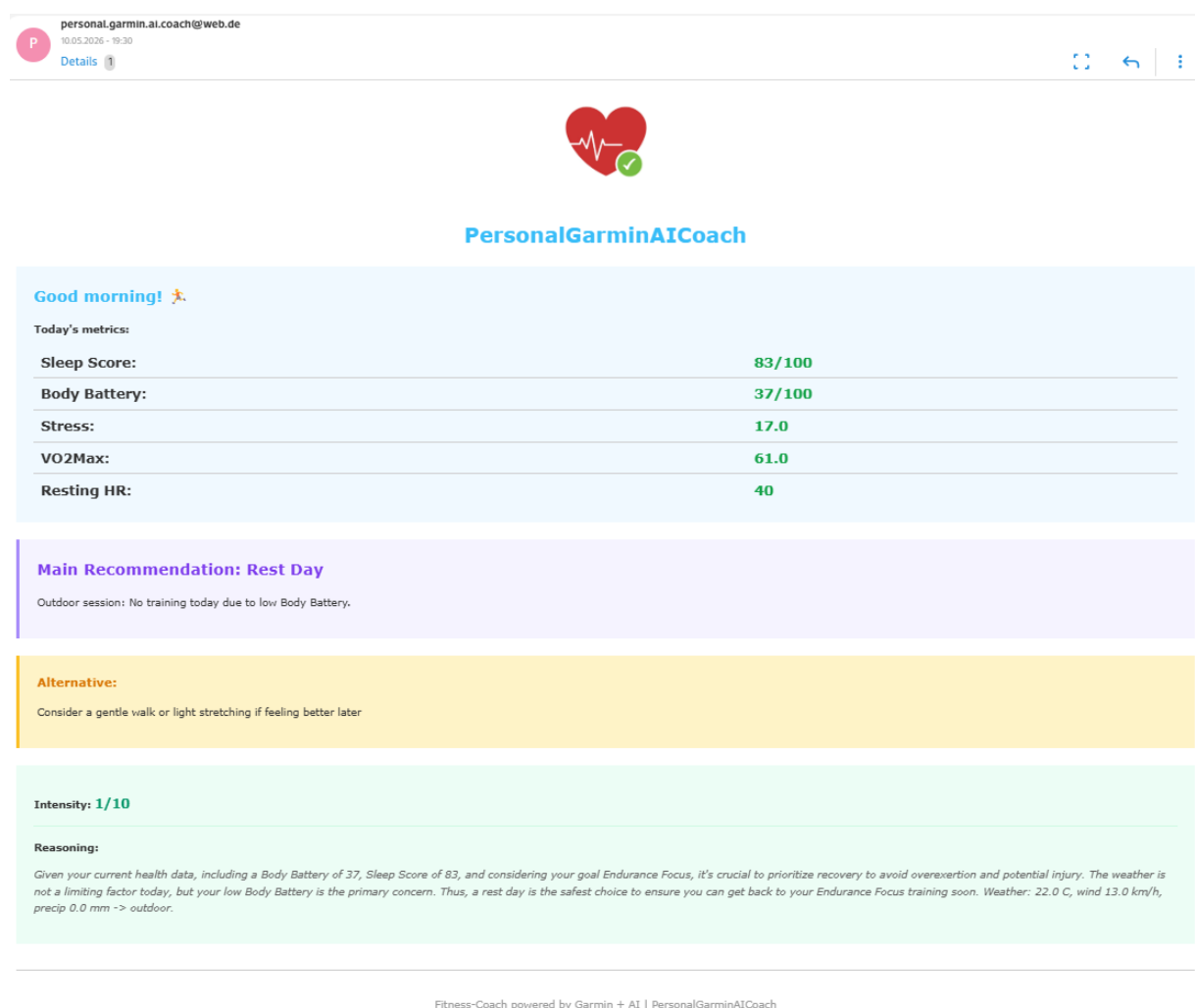


Abbildung 5: E-Mail Notification mit „Rest Day“

6.2 Deterministische Fallbacks

Fällt das LLM aus (fehlender API-Key, oder dreimaliger Provider-Fehler) würde das System ohne Sicherung keine Empfehlung liefern. Dafür existiert ein regelbasierter Ersatz-Coach (`_concrete_next_training`), der ohne LLM eine sinnvolle Empfehlung erzeugt.

Der Ablauf von `generate_coach_recommendation` ist:

1. **Cache prüfen** (sofern kein [refresh](#)); bei gültigem Treffer sofort zurückgeben.
2. **LLM aufrufen** mit bis zu **3 Versuchen** (Retry nur bei wiederholbaren Fehlern, siehe Kapitel 8). Erfolg → source: "model".
3. **Lokaler Fallback**, wenn alle Versuche scheitern oder kein API-Key vorhanden ist → source: "local", inkl. `fallback_reason`.

Der Fallback wählt anhand von Ziel und Erholungszustand eine passende Vorlage aus einer Asset-Datei (coach_examples.json) (z. B. marathon_longrun, endurance_recovery, strength_session oder rest_day). Die Vorlagen enthalten Platzhalter (Sleep, Body Battery etc.), die mit den realen Tageswerten gefüllt werden.

6.3 Goal-basierte Intensität

Jedes Trainingsziel besitzt eine Basis-Intensität (`_calculate_goal_intensity_baseline`), die als Ausgangspunkt dient, bevor der Erholungszustand sie ggf. nach unten korrigiert:

Ziel	Basis-Intensität (1–10)
Strength Focus	9
Endurance Focus / Marathon	7
Build Strength and Endurance	7
Default (Fallback)	6

Tabelle 8: Goal-Intensität

Greift der Recovery-Modus (6.1), wird diese Basis unabhängig vom Ziel auf maximal 3–4 gedeckelt. So bleibt das Ziel handlungsleitend, ohne den Erholungsschutz auszuhebeln.

Zusätzlich korrigiert der Code Ziel-Bezeichnungen in der LLM-Antwort (`_fix_goal_references`): Nutzt das Modell ein Synonym statt des exakten Zielnamens, wird der Text automatisch auf den korrekten Namen zurückgeschrieben.

7 Wetter-Integration & Context-Awareness

Das System bezieht die aktuelle Wetterlage in jede Empfehlung ein. So kann der Coach z. B. bei Regen auf eine Indoor-Einheit umschwenken, obwohl die Erholungswerte ein hartes Training erlauben würden.

7.1 Open-Meteo Geocoding & Wetter

Die Wetter-Integration nutzt zwei kostenlose Open-Meteo-Endpunkte und benötigt keinen API-Key:

1. **Geocoding** (`geocoding-api.open-meteo.com/v1/search`): Im Dashboard-Seitenbereich gibt der Nutzer eine Stadt ein („Stadt suchen“). Der Treffer füllt Breiten- und Längengrad automatisch und speichert sie im Profil. Manuelle Koordinateneingabe bleibt als Fallback möglich.
2. **Forecast** (`api.open-meteo.com/v1/forecast`): Auf Basis der Koordinaten werden die aktuellen Werte abgefragt (`fetch_current_weather`):

Feld	Open-Meteo-Parameter	Bedeutung
temperature_c	temperature_2m	Temperatur in °C
wind_speed_kmh	wind_speed_10m	Windgeschwindigkeit in km/h
precipitation_mm	precipitation	Niederschlag in mm

Tabelle 9: Open-Meteo-Felder

Die Abfrage läuft mit einem Timeout von 8 Sekunden und der Zeitzone Europe/Berlin. Schlägt der Abruf fehl, gibt die Funktion None zurück und das System arbeitet ohne Wetterkontext weiter (siehe [Indoor-Default in 7.2](#)).

7.2 Indoor/Outdoor-Entscheidung

Aus den Wetterwerten leitet der Code (`_weather_environment`) deterministisch ab, ob die Hauptempfehlung outdoor oder indoor sein soll:

Bedingung	Environment
$5\text{ °C} \leq \text{Temperatur} \leq 35\text{ °C}$ und Niederschlag $\leq 20\text{ mm}$	outdoor
Temperatur außerhalb dieses Fensters oder Niederschlag $> 20\text{ mm}$	indoor
Wetterdaten fehlen / unvollständig	indoor (sicherer Default)

Tabelle 10: Indoor/Outdoor-Logik

Damit das Wetter konsistent in der Empfehlung erscheint, greifen drei Schritte (`_apply_weather_context`):

1. **Präfix:** Die Empfehlung beginnt explizit mit „Outdoor session:“ bzw. „Indoor session:“

2. **Konfliktbereinigung:** Sätze in der Begründung, die das gegenteilige Environment befrworten, werden entfernt (`_remove_conflicting_environment_claims`), so widerspricht sich die Begründung nicht selbst.
3. **Wetter-Snippet:** Ein Faktensatz mit den konkreten Werten wird angehängt, z. B. „*Weather: 18.8 C, wind 7.9 km/h, precip 0.0 mm -> outdoor.*“

8 Resilienz: Caching, Rate-Limitierung & Retry

Das System ist auf Dauerbetrieb ausgelegt und ruft mehrere externe Dienste mit eigenen Rate-Limits auf (Garmin, Groq, Open-Meteo). Damit es stabil bleibt und keine API überlastet, greifen drei unabhängige Mechanismen ineinander: ein gestaffeltes Caching, ein persistierter Backoff für Garmin und eine Retry-Logik für das LLM.

8.1 Drei getrennte Caching-Ebenen

Ebene	Lebensdauer / Intervall	Zweck
Empfehlung	6 Stunden (pro Nutzer und Sprache)	teure LLM-Aufrufe vermeiden
Wetter	alle 10 Minuten, solange das Dashboard offen ist	aktueller Umweltkontext ohne Spam
Garmin-Rohdaten	nur bei aktivem Backoff aus dem Cache	Garmin-Rate-Limit schonen

Tabelle 11: drei Caching-Ebenen

8.2 Persistierter Backoff für Garmin

Garmin sperrt bei zu häufigen Anfragen temporär (HTTP 429). Statt blind weiter anzufragen, speichert das System nach jedem Fehlschlag den nächsten erlaubten Abrufzeitpunkt und staffelt die Wartezeit zunehmend (eine Stunde, vier Stunden, 24 Stunden, danach 48 Stunden). Solange dieser Zeitraum aktiv ist, wird der Live-Abruf übersprungen und auf die zuletzt gespeicherten Daten zurückgegriffen. Dieser Zustand überlebt einen Neustart, da er auf Platte abgelegt wird. Details dazu finden sich in [Kapitel 4.1](#).

8.3 Retry für das LLM

Beim LLM-Aufruf unterscheidet das System zwischen wiederholbaren und endgültigen Fehlern. Wiederholbar sind etwa Überlastung (429), nicht verfügbarer Dienst (503), Timeouts oder

eine leere bzw. unparsbare Antwort. Endgültig ist hingegen ein aufgebrauchtes Kontingent, das ein erneuter Versuch nicht beheben würde.

Der Ablauf:

1. Bis zu drei Versuche, das Modell aufzurufen und die Antwort zu parsen.
2. Bei einem wiederholbaren Fehler wird kurz gewartet (eine Sekunde beim ersten, zwei beim zweiten Versuch) und erneut versucht.
3. Bei einem endgültigen Fehler oder nach dem dritten Fehlversuch greift der lokale Fallback aus [Kapitel 6.2](#).

So bleibt das System auch bei kurzzeitigen Störungen des LLM-Anbieters reaktionsfähig und liefert immer eine Empfehlung, entweder vom Modell, aus dem Cache oder vom regelbasierten Fallback.

9 Sicherheit & Datenschutz

Sobald ein Agent mit Gesundheitsdaten und Zugangsdaten arbeitet, wird Sicherheit zum kritischsten Baustein. Dieses Kapitel beschreibt, wie das System Garmin-Credentials schützt, welche Daten bewusst lokal liegen und welche Risiken daraus folgen.

9.1 HashiCorp Vault (KV v2)

Garmin-Zugangsdaten werden nicht im Klartext in der Anwendung gehalten, sondern in HashiCorp Vault OSS über die KV-v2-Engine abgelegt. Vault wird über drei Umgebungsvariablen angebunden: die Adresse der Instanz, ein Zugriffstoken und der KV-Pfad.

Der Pfad unterstützt eine Nutzer-Platzhaltersyntax. Steht im konfigurierten Pfad der Platzhalter für die Nutzer-ID, wird dieser zur Laufzeit durch die dateisystemsicher bereinigte Nutzer-ID ersetzt. So erhält jeder Nutzer einen eigenen, isolierten Geheimnis-Pfad.

Variable	Bedeutung	Default
VAULT_ADDR	Adresse der Vault-Instanz	nicht gesetzt
VAULT_TOKEN	Zugriffstoken	nicht gesetzt
VAULT_KV_PATH	KV-Pfad, optional mit Nutzer-Platzhalter	kv/garmin/default

Tabelle 12: Umgebungsvariablen der Vault-Anbindung

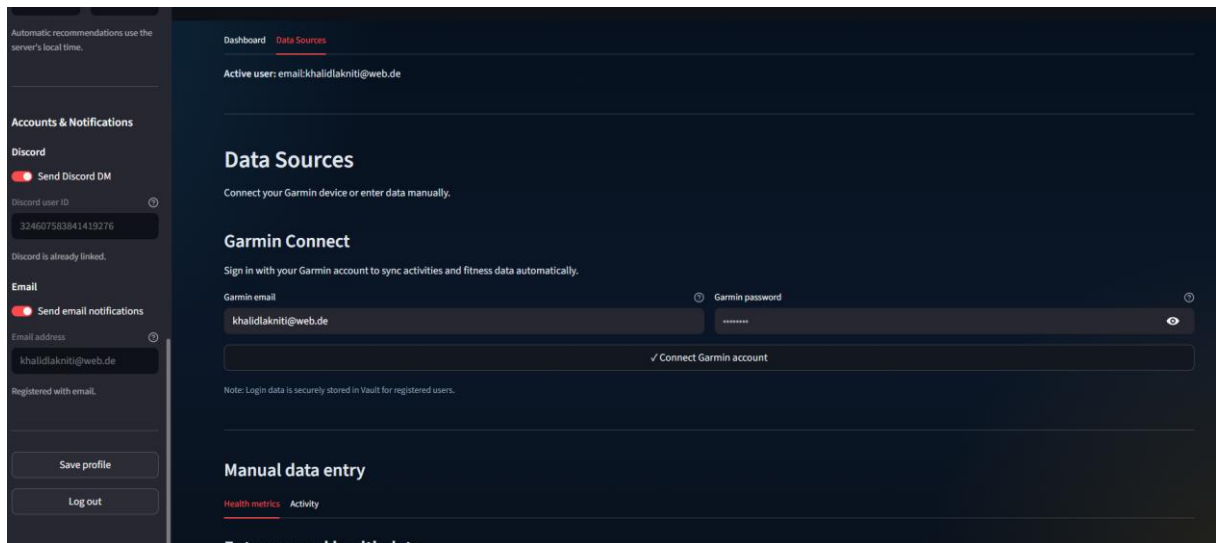


Abbildung 6: Garmin & Vault

Lesen und Schreiben erfolgen über die HTTP-API von Vault mit dem Token im Anfrage-Header und einem Timeout von zehn Sekunden. Beim Schreiben verlangt die KV-v2-API, dass die eigentlichen Daten in einem Datenobjekt verschachtelt werden. In Log-Ausgaben wird das Token konsequent maskiert, damit es nicht versehentlich in Protokolldateien landet.

Bemerkenswert für die Sicherheitsarchitektur: Das Speichern der Credentials geschieht ausschließlich gegen Vault. Schlägt der Vault-Schreibvorgang fehl, bricht der Vorgang mit einem Fehler ab, statt still auf eine unsichere lokale Ablage auszuweichen. Auch das Laden der über das Dashboard hinterlegten Credentials liest nur aus Vault.

9.2 Bedrohungsmodell & bewusste lokale JSON-Ablage

Nicht alle Daten liegen in Vault. Für ein ehrliches Cookbook ist die Trennung zwischen geschützten und ungeschützten Daten zentral.

Datenart	Ablageort	Schutz
Garmin-Zugangsdaten (über Dashboard)	HashiCorp Vault	verschlüsselt, tokenbasiert
API-Keys und Tokens	.env-Datei	nicht eingeecheckt, Klartext auf dem Host
Gesundheitsmetriken und Aktivitäten	lokale JSON-Dateien	Dateisystemrechte

Nutzerprofile	lokale JSON-Dat teien	Dateisystemrechte
---------------	--------------------------	-------------------

Tabelle 13: Ablageorte und Schutzniveau der verschiedenen Datenarten

Daraus ergeben sich folgende bewusste Designentscheidungen und Restrisiken:

Die API-Keys in der .env liegen als Klartext auf dem Host. Sie sind zwar von der Versionsverwaltung ausgeschlossen, aber wer Zugriff auf den Server hat, kann sie lesen. Eine weitergehende Absicherung würde bedeuten, auch diese Secrets in Vault zu verwalten.

Die Gesundheitsdaten liegen unverschlüsselt als JSON im Dateisystem. Das ist ein bewusster Kompromiss zugunsten von Einfachheit und Nachvollziehbarkeit. Der Schutz beschränkt sich auf die Dateisystemrechte des Hosts. Bei einem Mehrnutzerbetrieb mit echten Personendaten wäre eine Verschlüsselung im Ruhezustand oder eine Datenbank mit Zugriffskontrolle der nächste logische Schritt.

Als Fazit lässt sich festhalten: Ein Agent ist nur so vertrauenswürdig wie der Schutz der Daten, mit denen er arbeitet. Das System schützt das sensibelste Gut, die Garmin-Zugangsdaten, mit einem etablierten Geheimnis-Manager und macht bei den übrigen Daten den Kompromiss transparent, statt ihn zu verschleiern.

10 Benutzeroberfläche & Scheduling

Die Anwendung wird vollständig über ein Streamlit-Dashboard bedient. Es dient gleichzeitig als Onboarding-Portal, Konfigurationszentrale und Laufzeitumgebung für den automatischen Scheduler.

10.1 Streamlit-Dashboard

Das Dashboard ist in einen Seitenbereich für Profil und Einstellungen sowie einen Hauptbereich mit Dashboard- und Datenquellen-Ansichten gegliedert. Die wichtigsten Funktionsblöcke sind:

Bereich	Funktion
Access und Profile	Mobilität, Trainingsziel, freie Notizen, Standort per Stadtsuche, Sprache
Coach	Garmin-Daten manuell aktualisieren, neue Empfehlung anfordern, Status anzeigen

Automatic Recommendations	Automatik aktivieren, zwei Tageszeiten festlegen
Accounts und Notifications	Discord-DM und E-Mail-Benachrichtigungen konfigurieren
Data Sources	Garmin-Credentials hinterlegen, manuelle Gesundheits- und Aktivitätsdaten eingeben

Tabelle 14: Funktionsblöcke des Dashboards

Die Sprachumschaltung zwischen Englisch und Deutsch wird pro Nutzerprofil gespeichert und gilt sowohl für die Oberfläche als auch für die generierten Empfehlungen.



Abbildung 7: Dashboard Overview

10.2 Background-Worker-Thread (60s-Loop)

Die Automatisierung läuft als Hintergrund-Thread innerhalb des Streamlit-Prozesses. Beim Start wird genau ein Scheduler als Daemon-Thread gestartet, der in einer Endlosschleife alle 60 Sekunden prüft, ob für einen Nutzer eine konfigurierte Zeit fällig ist.

Der Ablauf je Durchlauf:

1. Über alle bekannten Nutzer iterieren und nur diejenigen mit aktivierter Automatik betrachten.

2. Für jeden Nutzer die zwei konfigurierten Tageszeiten gegen die letzte Ausführung prüfen. Fällig ist eine Zeit, wenn die aktuelle Serverzeit den Zeitpunkt erreicht hat und an diesem Tag noch keine Ausführung erfolgt ist.
3. Bei Fälligkeit die Kette ausführen: Garmin-Daten aktualisieren, Empfehlung erzeugen, Benachrichtigung versenden.
4. Den Ausführungszeitpunkt pro Zeit speichern, damit derselbe Slot nicht erneut auslöst.

Eine nicht blockierende Sperre stellt sicher, dass sich parallele Durchläufe nicht überschneiden. Alle Zeiten beziehen sich auf die lokale Serverzeit. Da der Scheduler im Streamlit-Prozess lebt, läuft die Automatik nur, solange die Anwendung läuft.

10.3 Onboarding & 6-stellige Verifizierung

Der Zugang ist durch eine Verifizierung geschützt. Neue Nutzer registrieren sich entweder per E-Mail oder per Discord. Der Ablauf:

1. Kontaktmethode wählen und Adresse eingeben.
2. Das System erzeugt einen sechsstelligen Code als Zufallszahl im Bereich von 100000 bis 999999.
3. Der Code wird über den gewählten Kanal zugestellt, per E-Mail oder Discord.
4. Nach Eingabe des korrekten Codes wird das Konto verifiziert und das Dashboard freigeschaltet.

Die Nutzerdaten und der jeweils ausstehende Code werden in einer zentralen Nutzerdatei gehalten. Beim Zugriff auf diese Datei wird eine Dateisperre verwendet, um gleichzeitige Schreibvorgänge abzusichern. Über dasselbe Verfahren lässt sich später eine zweite Kontaktmethode mit dem bestehenden Konto verknüpfen.

Sign In / Verification ↗

Choose a sign-in method: email or Discord.

Choose sign-in method

☒ Email ☐ Discord

Email sign-in with a clear separation between login and registration.

[I already have an account](#) [Register a new account](#)

Email

Password

Sign in with email and password

Sign In / Verification

Choose a sign-in method: email or Discord.

Choose sign-in method

☒ Email ☐ Discord

Email sign-in with a clear separation between login and registration.

[I already have an account](#) [Register a new account](#)

Email for registration

Password for new email sign-in

Register & send code by email

Verification code

Verify code (email)

Abbildung 8: Login/Register Screen

Your verification code for PersonalGarminAICoach

 **personal.garmin.ai.coach@web.de**
an mich ▼

Your verification code: **390887**

Abbildung 9: Registration Code via E-Mail

11 Deployment & Betrieb

Das System ist für den dauerhaften Betrieb auf einem Linux-Server ausgelegt. Es kombiniert ein schlankes Docker-Image mit einem Systemd-Dienst, der für Autostart und automatische Neustarts sorgt.

11.1 Docker

Das Image basiert auf einem schlanken Python-3.11-Image. Der Build installiert zunächst nur die Abhängigkeiten und kopiert anschließend den Quellcode, was die Schichten-Zwischenspeicherung effizient nutzt. Der Container exponiert Port 8080 und startet Streamlit direkt als Hauptprozess.

Aspekt	Wert
Basis-Image	python:3.11-slim
Exponierter Port	8080
Startkommando	streamlit run web/app.py auf Port 8080, Adresse 0.0.0.0
Umgebungsvariablen	PYTHONDONTWRITEBYTECODE und PYTHONUNBUFFERED gesetzt

Tabelle 15: Eckdaten des Docker-Images

Die Bindung an die Adresse 0.0.0.0 ist notwendig, damit der Dienst von außerhalb des Containers erreichbar ist. Die Secrets werden nicht ins Image gebacken, sondern zur Laufzeit über die Umgebung bereitgestellt.

11.2 Systemd und Betrieb in der Cloud

Im Produktivbetrieb läuft die Anwendung auf einer Ubuntu-Instanz in der Open Telekom Cloud. Statt eines Containers wird dort ein Systemd-Dienst verwendet, der dieselbe Streamlit-Anwendung direkt aus der virtuellen Umgebung startet.

Wesentliche Eigenschaften des Dienstes:

Der Dienst wartet auf eine verfügbare Netzwerkverbindung, bevor er startet. Die Secrets werden über eine Umgebungsdatei geladen, sodass keine Zugangsdaten in der Dienstdefinition stehen. Standardausgabe und Fehlerausgabe werden in das Journal geschrieben, was die Fehlersuche über die übliche Logabfrage ermöglicht.

Für die Verfügbarkeit sorgt eine Neustart-Richtlinie: Der Prozess wird bei einem Fehler automatisch neu gestartet, mit einer kurzen Wartezeit zwischen den Versuchen. Damit überlebt die Anwendung sowohl Abstürze als auch einen Neustart des Servers, da der Dienst beim Systemstart automatisch hochfährt.

Einstellung	Wert
Startkommando	Streamlit auf Port 8080, Adresse 0.0.0.0

Umgebungsdatei	.env im Projektverzeichnis
Neustart	always, mit kurzer Wartezeit
Logging	System-Journal
Autostart	aktiv beim Systemstart

Tabelle 16: Eckdaten des Systemd-Dienstes

Da der in Kapitel 10 beschriebene Scheduler innerhalb des Streamlit-Prozesses läuft, genügt dieser eine Dienst, um sowohl das Dashboard bereitzustellen als auch die automatischen Empfehlungen auszulösen. Ein separater Hintergrunddienst für die Zeitplanung ist nicht erforderlich.

12 Testing & Lokaler Trockenlauf

Da das System mit echten externen Diensten und personenbezogenen Daten arbeitet, ist ein gefahrloser lokaler Trockenlauf wichtig. Dieses Kapitel zeigt, wie sich einzelne Komponenten ohne vollständige Infrastruktur prüfen lassen.

12.1 Coach-Logik isoliert ausführen

Der Coach Agent lässt sich direkt von der Kommandozeile aufrufen, ohne das Dashboard zu starten. So kann die Empfehlungserzeugung mit beliebigem Profil getestet werden.

Option	Bedeutung
run-model	ruft das LLM direkt auf, statt nur den Prompt zu bauen
mobility	setzt das Mobilitätsprofil
preference	setzt die Trainingspräferenz
goal	setzt das Trainingsziel
model	überschreibt das verwendete Groq-Modell
refresh	umgeht den Cache und erzwingt eine neue Empfehlung

Tabelle 17: Kommandozeilenoptionen des Coach Agent

Ein typischer Aufruf erzeugt eine Empfehlung für ein bestimmtes Ziel und ignoriert dabei den Cache. Ohne gesetzten Groq-Schlüssel liefert der Agent automatisch den lokalen Fallback aus Kapitel 6, sodass sich auch die Guardrails ohne echten LLM-Zugang prüfen lassen.

12.2 Garmin-Abruf gezielt testen

Der Garmin-Abruf ist als eigenständiges Skript gestaltet und nutzt eigene Optionen. Damit lässt sich der Datenabruf für einen einzelnen Nutzer auslösen und das Verhalten bei aktivem Backoff kontrollieren.

Option	Bedeutung
user-id	begrenzt den Abruf auf einen bestimmten Nutzer
debug	aktiviert ausführliche Protokollausgaben
force	ignoriert den Backoff und erzwingt einen Abrufversuch

Tabelle 18: Kommandozeilenoptionen des Garmin-Abrufs

Die ausführliche Protokollausgabe macht die einzelnen Schritte des Logins sichtbar und unterscheidet die Fehlerklassen aus Kapitel 4. So lässt sich etwa ein Rate-Limit von einem Authentifizierungsfehler trennen.

12.3 Manuelle Testwerte im Dashboard

Für Tests ohne echtes Garmin-Konto bietet das Dashboard die manuelle Dateneingabe. Damit lassen sich Gesundheitsmetriken und Aktivitäten von Hand setzen, die mit der Quelle als manuell markiert werden. Auf diese Weise können gezielt Grenzfälle erzeugt werden, etwa eine niedrige Body Battery, um den Rest-Day-Pfad auszulösen, oder bestimmte Wetterwerte, um die Indoor-Entscheidung zu prüfen.

12.4 Stand der automatisierten Tests

Im aktuellen Stand des Projekts ist die Testabdeckung gering. Die Projektdokumentation weist auf einen Test der Nutzerverwaltung, ein entsprechendes Testverzeichnis ist im Repository jedoch nicht vorhanden. Die Verifizierung erfolgt daher überwiegend manuell über die oben beschriebenen Trockenläufe.

Für einen produktiven Ausbau wäre der Aufbau einer echten Testsuite der nächste Schritt. Sinnvolle Kandidaten sind die deterministischen Bausteine, die sich ohne externe Dienste prüfen lassen: die Indoor- und Outdoor-Entscheidung, die Recovery-Schwellen, die Backoff-Staffelung und das defensive Parsen der LLM-Antwort.

13 Fazit und Lessons Learned

13.1 Zusammenfassung

Mit dem Personal Garmin AI Coach wurde eine funktionale und praxistaugliche Toolchain realisiert, die den Weg von biometrischen Rohdaten zu einer personalisierten Handlungsempfehlung autonom überbrückt. Das System verbindet die Datenbeschaffung aus Garmin Connect, die Anreicherung mit Wetterdaten und die Entscheidungsfindung durch ein Large Language Model zu einem geschlossenen Kreislauf.

Den eigentlichen Mehrwert liefert nicht das Sprachmodell allein, sondern dessen Einbettung in deterministische Sicherheitsregeln. Erholungsschwellen, eine zielbasierte Intensität und ein regelbasierter Fallback stellen sicher, dass das System auch bei kritischen Werten oder einem Ausfall des Anbieters eine sinnvolle und sichere Empfehlung liefert. Die Integration eines Geheimnis-Managers und die Berücksichtigung von Umweltfaktoren heben das Projekt über einen reinen Machbarkeitsnachweis hinaus.

13.2 Lessons Learned

Im Verlauf der Entwicklung konnten mehrere zentrale Erkenntnisse gewonnen werden.

Struktur schlägt Kreativität. Sprachmodelle neigen bei unstrukturierten Daten zu Halluzinationen. Die größte Hürde war nicht die Generierung des Textes, sondern das Erzwingen einer verlässlich maschinenlesbaren Ausgabe. Erst der erzwungene JSON-Modus, ein defensives Parsen der Antwort und eine konsequente Normalisierung der Felder machten die nachgelagerte Automatisierung über Discord und E-Mail robust.

Vertrauen ist gut, Kontrolle ist besser. Ein KI-Coach, der Gesundheitsempfehlungen ausspricht, darf sich nicht allein auf das Modell verlassen. Die Verlagerung der sicherheitskritischen Entscheidungen in deterministischen Code war die wichtigste Designentscheidung des Projekts. Das Modell formuliert, aber die Regeln entscheiden über die zulässige Intensität.

API-Ökonomie und Resilienz. Das Rate-Limiting der Drittanbieter ist eine kritische Designvariable. Die Trennung der Caching-Ebenen, ein persistierter und gestaffelter Backoff für Garmin sowie eine Retry-Logik für das Sprachmodell waren essenziell, um die Stabilität im Dauerbetrieb sicherzustellen.

Sicherheit ist die Grundlage des Vertrauens. Sobald ein Agent mit persönlichen Gesundheitsdaten und Zugangsdaten arbeitet, wird das Geheimnis-Management zum kritischsten Baustein.

Die Ablage der Zugangsdaten in einem etablierten Geheimnis-Manager und die transparente Einordnung der übrigen, lokal gehaltenen Daten waren bewusste Entscheidungen.

KI-gestützte Entwicklung verschiebt den Schwerpunkt. Der Einsatz KI-gestützter Programmierung beschleunigt die Erstellung von Standardcode erheblich. Die eigentliche Ingenieursleistung verschiebt sich dadurch weg vom reinen Schreiben der Syntax hin zur Orchestrierung der Toolchain und zur Absicherung der Schnittstellen.

13.3 Ausblick

Zukünftige Erweiterungen könnten an mehreren Stellen ansetzen.

Ein aktiver Feedback-Loop wäre der natürliche nächste Schritt. Bewertet der Nutzer die Empfehlung als zu hart oder zu leicht, könnte das System diese Rückmeldung in der Nutzerhistorie speichern und über die Zeit ein individuelles Belastungsprofil lernen, um die Empfehlungen präziser zu kalibrieren.

Auf der technischen Seite würde der Aufbau einer automatisierten Testsuite die Wartbarkeit deutlich erhöhen, insbesondere für die deterministischen Bausteine wie die Recovery-Schwellen, die Indoor- und Outdoor-Entscheidung und die Backoff-Staffelung.

Im Bereich Sicherheit wäre die Verschlüsselung der lokal gehaltenen Gesundheitsdaten im Ruhezustand oder der Umstieg auf eine Datenbank mit Zugriffskontrolle ein sinnvoller Ausbau, sobald das System für mehrere Nutzer mit echten Personendaten produktiv betrieben wird.

14 Screenshots

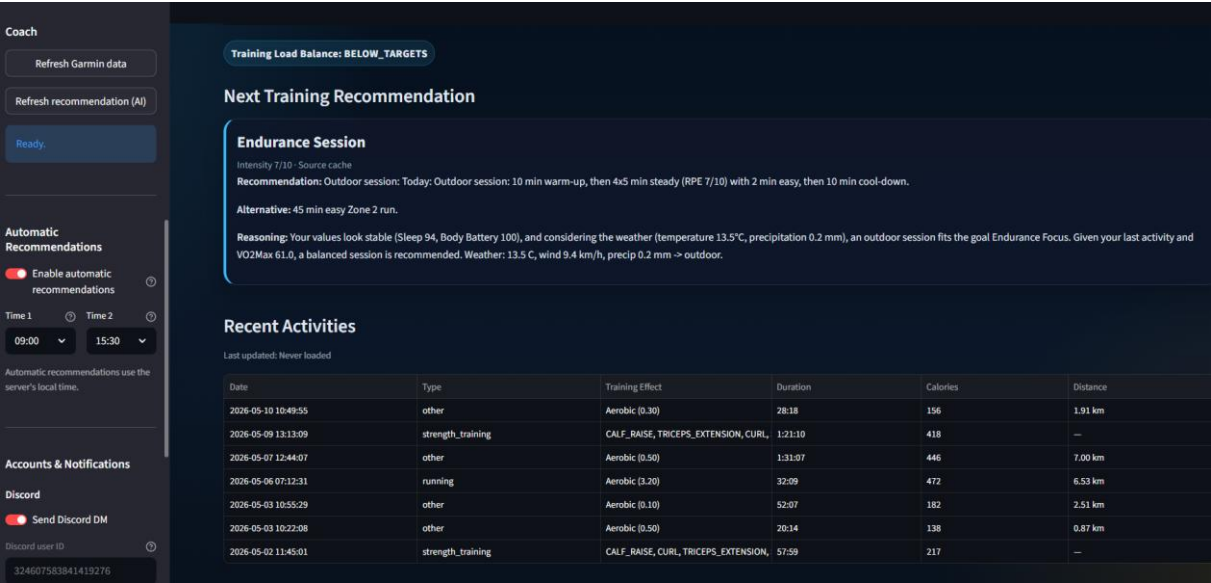


Abbildung 10: Recent Activities



PersonalGarminAICoach

APP

9:02 AM

May 11, 2026

PersonalGarminAICoach ·  / 

 English

Title: Endurance Session

Main Recommendation

Today: Outdoor session: 10 min warm-up, then 4x5 min steady (RPE 7/10) with 2 min easy, then 10 min cool-down.

Alternative

45 min easy Zone 2 run.

Reasoning

Your values look stable (Sleep 94, Body Battery 100), and considering the weather (temperature 13.5°C, precipitation 0.2 mm), an outdoor session fits the goal Endurance Focus. Given your last activity and VO2Max 61.0, a balanced session is recommended.

 Deutsch

Title: Ausdauersitzung

Main Recommendation

Heute: Outdoor-Sitzung: 10 min Aufwärmen, dann 4x5 min Steady (RPE 7/10) mit 2 min leicht, dann 10 min Abkühlen.

Alternative

45 min einfache Zone-2-Lauf.

Reasoning

Ihre Werte sehen stabil aus (Schlaf 94, Body-Battery 100), und unter Berücksichtigung des Wetters (Temperatur 13,5°C, Niederschlag 0,2 mm), passt eine Outdoor-Sitzung zum Ziel Ausdauerfokus. Angesichts Ihrer letzten Aktivität und VO2Max 61,0 wird eine ausgewogene Sitzung empfohlen.

Sleep Score	Body Battery	Stress
94/100	100/100	9.0
VO2Max	RHR	Intensity
61.0	41	7/10

Garmin + AI · PersonalGarminAICoach · Today at 9:01 AM

Abbildung 11: Discord Embed